IT112 Web System Development

Phase 2 – Backend Development

Lab Report by TechNet

Group Members:

Mico Malana

Christian Silo

Jay Lloyd Chicogo

John Nelson Andes

Allen Benedict Asuncion

Submitted to:

Guillermo V. Red, Jr., DIT

I. Overview of Backend Setup

The **TaskWise Backend** follows a modular, feature-based architecture using Node.js, Express, and MongoDB. It is designed for scalability, maintainability, and clear separation of concerns.

II. Code Snippets

A. server.js

```
const express = require("express");
require("dotenv").config();
const cors = require("cors");

const connectDB = require("./config/db");

const studentRouter = require("./modules/routes/student.router");
const groupRouter = require("./modules/routes/group.router");
const taskRouter = require("./modules/routes/task.router");

const app = express();

const routers = [studentRouter, groupRouter, taskRouter];

app.use(express.json());
app.use(cors());

routers.forEach((router) => {
  app.use("/api", router);
});

async function start() {
  const port = process.env.PORT;
  const uri = process.env.MONGO_URI_LOCAL;

  await connectDB(uri);

  app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
  });
}

start();
```

B. Sample Model – task.model.js

```
const mongoose = require("mongoose");
const taskSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      maxlength: 20,
    },
    difficulty: {
      type: Number,
      enum: [1, 2, 3, 4, 5],
    },
    deadline: {
      type: Date,
      required: true,
    },
    subject: {
      type: String,
      maxlength: 20,
    },
    creator: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Student",
    },
  },
  { timestamps: true },
);

const Task = mongoose.model("Task", taskSchema);

module.exports = Task;
```

C. Sample Routes – task.router.js

```
const express = require("express");

const {
  getTasks,
  createTask,
  updateTask,
  deleteTask,
} = require("../controllers/task.controller");

const taskRouter = express.Router();

const authMiddleware = require("../middlewares/auth.middleware");
```

```
// GET ALL THE TASKS
taskRouter.get("/task", authMiddleware, async (req, res) => {
  const { id } = req.student;
  try {
    const tasks = await getTasks(id);

    res.status(200).json({ tasks });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// CREATE TASK
taskRouter.post("/task/create", authMiddleware, async (req, res)
=> {
  const task = req.body;
  const { id } = req.student;

  try {
    const newTask = await createTask(task, id);
    res.status(201).json({ newTask });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// UPDATE TASK
taskRouter.patch("/task/update/:id", async (req, res) => {
  try {
    const allowedFields = ["name", "difficulty", "deadline"];
    const task = {};

    for (let key of allowedFields) {
      if (req.body[key] !== undefined) {
        task[key] = req.body[key];
      }
    }

    const newTask = await updateTask(req.params.id, task);

    if (!newTask) {
      return res.status(404).json({ message: "Task not found" });
    }

    res.status(200).json({ newTask });
  } catch (error) {
```

```
    res.status(500).json({ message: error.message });
  }
});

taskRouter.delete("/task/delete/:id", authMiddleware, async (req,
res) => {
  const { id } = req.student;

  try {
    const deletedTask = await deleteTask(req.params.id, id);

    res.status(200).json({ deletedTask });
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

module.exports = taskRouter;
```

D. Sample Controller – task.controller.js

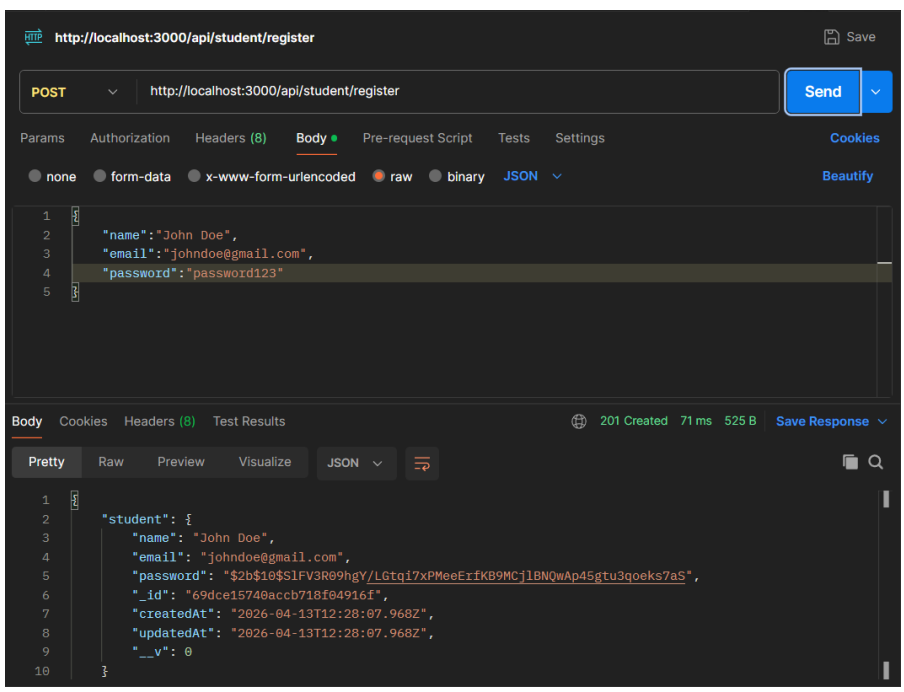
```
const Task = require("../models/task.model");
const mongoose = require("mongoose");

const getTasks = async (id) => {
  return await Task.find({ creator: new
mongoose.Types.ObjectId(id) });
};

const createTask = async (task, id) => {
  return await Task.create({
    ...task,
    creator: new mongoose.Types.ObjectId(id),
  });
};

const updateTask = async (taskId, task) => {
  return await Task.findByIdAndUpdate(
    taskId,
    { $set: task },
    {
      new: true,
      runValidators: true,
    },
  );
};
```

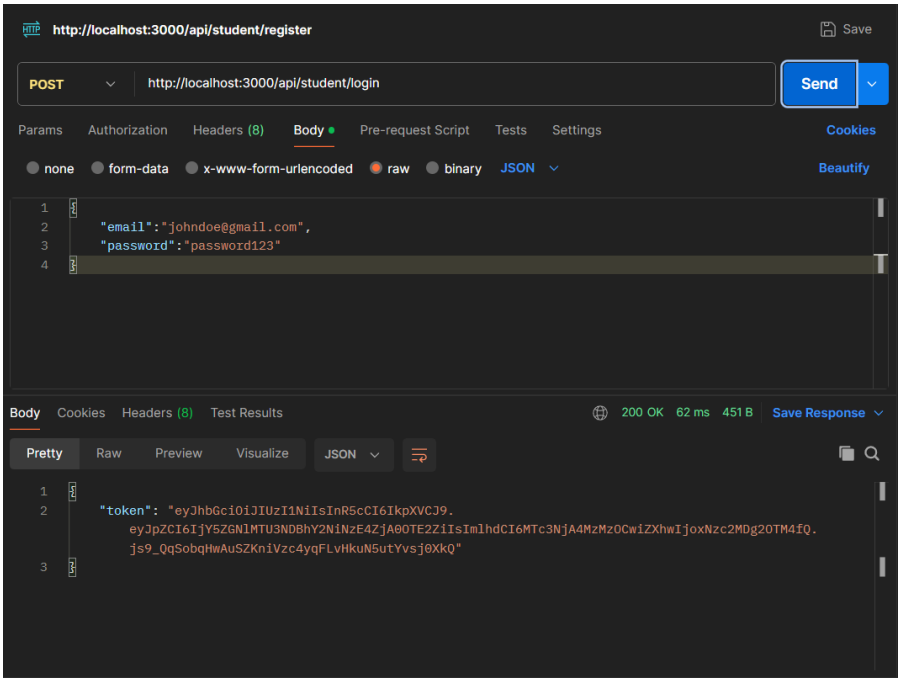
```
const deleteTask = async (taskId, studentId) => {
  const task = await Task.findOne({
    _id: new mongoose.Types.ObjectId(taskId),
    creator: new mongoose.Types.ObjectId(studentId),
  });

  if (!task) throw new Error("Task not found");

  await Task.findByIdAndDelete(taskId);

  return task;
};

module.exports = { getTasks, createTask, updateTask, deleteTask
};
```

III. API Testing

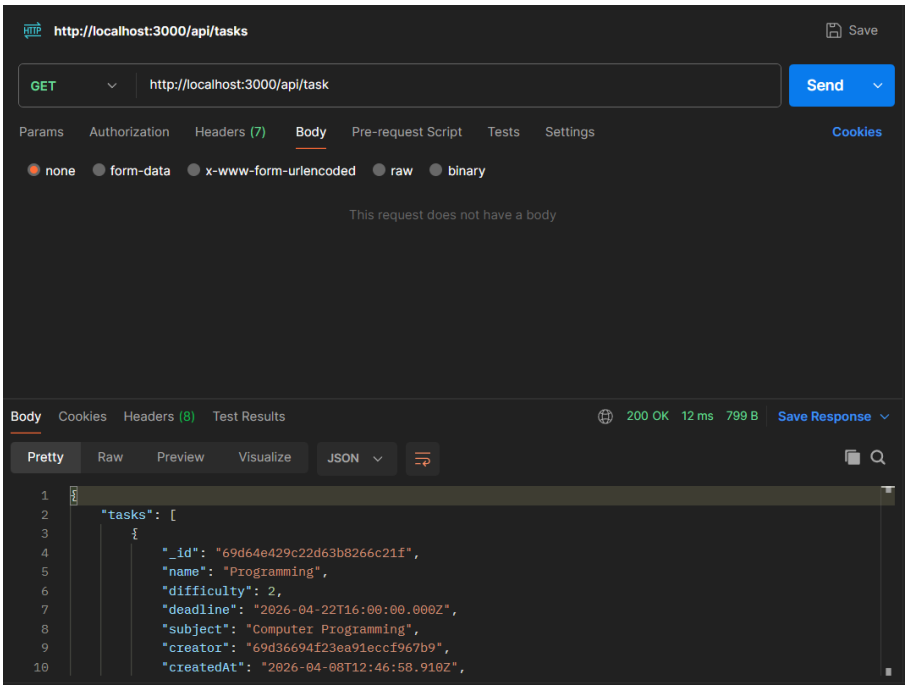
All routes were thoroughly tested using Postman, and the corresponding test cases are documented below.

A. Student




http://localhost:3000/api/student/register
Save
POST
http://localhost:3000/api/student/login
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"email":"johndoe@gmail.com",
"password":"password123"
}
Body
Cookies
Headers (8)
Test Results
200 OK
62 ms
451 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjY5ZGNlMTU3NDBhY2NiNzE4ZjA0OTE2ZiIsImlhdCI6MTc3NjA4MzMzOCwiZXhwIjoxNzc2MDg2OTM4fQ.
js9_QqSobqHwAuSZKniVzc4yqFLvHkuN5utYvsj0XkQ"
}

B. Task


http://localhost:3000/api/tasks
Save
GET
http://localhost:3000/api/task
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
This request does not have a body
Body
Cookies
Headers (8)
Test Results
200 OK
12 ms
799 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"tasks": [
{
"_id": "69d64e429c22d63b8266c21f",
"name": "Programming",
"difficulty": 2,
"deadline": "2026-04-22T16:00:00.000Z",
"subject": "Computer Programming",
"creator": "69d36694f23ea91eccf967b9",
"createdAt": "2026-04-08T12:46:58.910Z",

http://localhost:3000/api/task/create

Save

POST http://localhost:3000/api/task/create Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
    "name":"Test",
    "difficulty":1,
    "deadline":"04-27-26",
    "subject":"Mathematics"
}
```

Body Cookies Headers (8) Test Results 201 Created 16 ms 534 B Save Response

Pretty Raw Preview Visualize JSON

```
{
    "newTask": {
        "name": "Test",
        "difficulty": 1,
        "deadline": "2026-04-26T16:00:00.000Z",
        "subject": "Mathematics",
        "creator": "69d36694f23ea91eccf967b9",
        "_id": "69dce29140accb718f049176",
        "createdAt": "2026-04-13T12:33:21.877Z",
        "updatedAt": "2026-04-13T12:33:21.877Z",
```

http://localhost:3000/api/task/create

Save

PATCH http://localhost:3000/api/task/update/69dce29140accb718f049176 Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

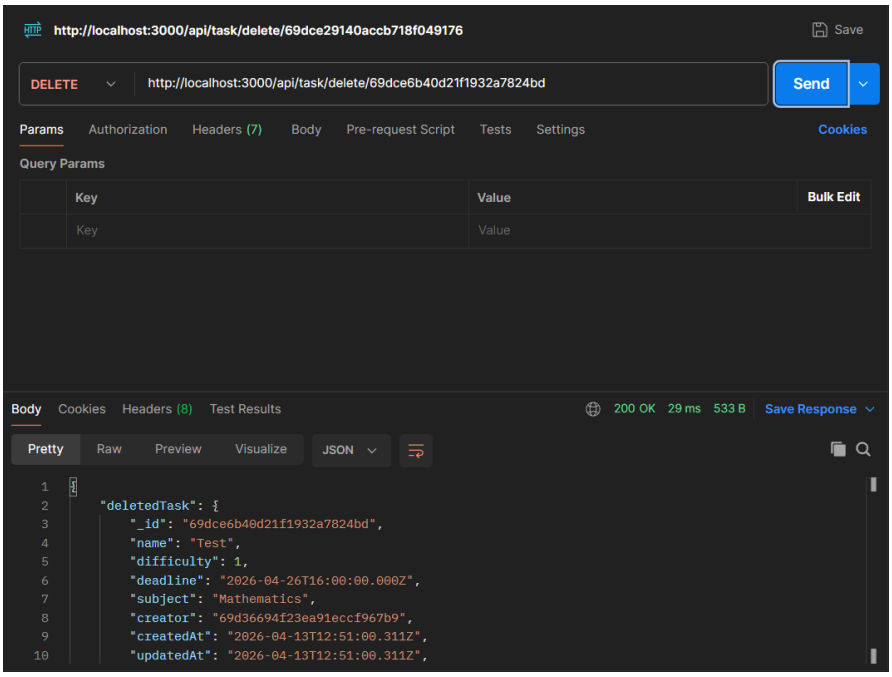
none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
    "difficulty":4
}
```

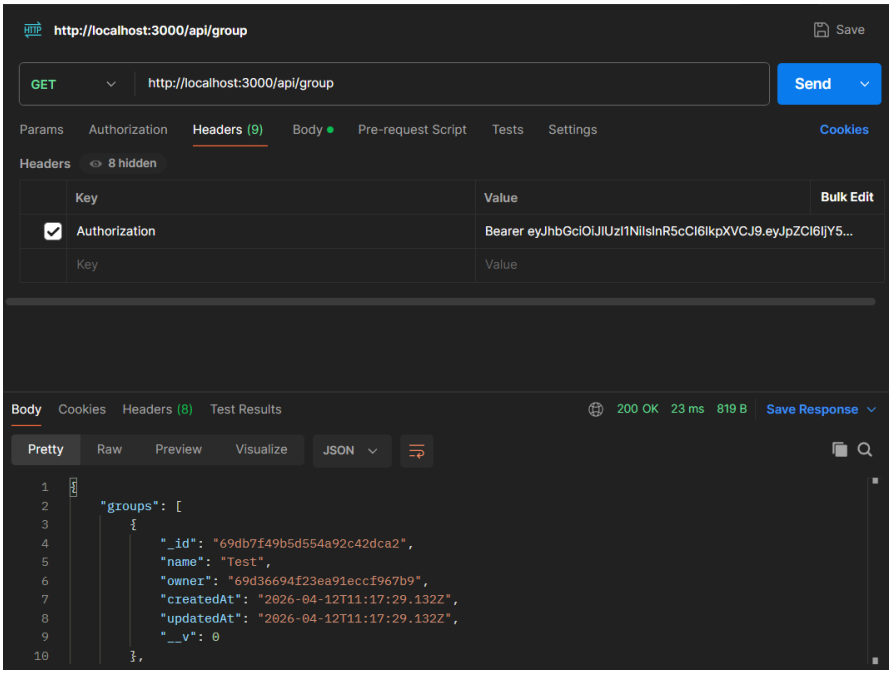
Body Cookies Headers (8) Test Results 200 OK 22 ms 529 B Save Response

Pretty Raw Preview Visualize JSON

```
{
    "newTask": {
        "_id": "69dce29140accb718f049176",
        "name": "Test",
        "difficulty": 4,
        "deadline": "2026-04-26T16:00:00.000Z",
        "subject": "Mathematics",
        "creator": "69d36694f23ea91eccf967b9",
        "createdAt": "2026-04-13T12:33:21.877Z",
        "updatedAt": "2026-04-13T12:35:02.928Z",
```


http://localhost:3000/api/task/delete/69dce29140accb718f049176
Save
DELETE
http://localhost:3000/api/task/delete/69dce6b40d21f1932a7824bd
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Bulk Edit
Key
Value
Body
Cookies
Headers (8)
Test Results
200 OK
29 ms
533 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"deletedTask": {
"_id": "69dce6b40d21f1932a7824bd",
"name": "Test",
"difficulty": 1,
"deadline": "2026-04-26T16:00:00.000Z",
"subject": "Mathematics",
"creator": "69d36694f23ea91eccf967b9",
"createdAt": "2026-04-13T12:51:00.311Z",
"updatedAt": "2026-04-13T12:51:00.311Z",

C. Group


http://localhost:3000/api/group
Save
GET
http://localhost:3000/api/group
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
Headers
8 hidden
Key
Value
Bulk Edit
Authorization
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5...
Key
Value
Body
Cookies
Headers (8)
Test Results
200 OK
23 ms
819 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"groups": [
{
"_id": "69db7f49b5d554a92c42dca2",
"name": "Test",
"owner": "69d36694f23ea91eccf967b9",
"createdAt": "2026-04-12T11:17:29.132Z",
"updatedAt": "2026-04-12T11:17:29.132Z",
"__v": 0
},

http://localhost:3000/api/group/create

Save

POST http://localhost:3000/api/group/create Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
    "name":"Project",
    "inviteCode":"RTY32H23G"
}
```

Body Cookies Headers (8) Test Results 201 Created 37 ms 470 B Save Response

Pretty Raw Preview Visualize JSON

```
{
    "name": "Project",
    "owner": "69d36694f23ea91eccf967b9",
    "inviteCode": "RTY32H23G",
    "_id": "69dce0bc40accb718f049169",
    "createdAt": "2026-04-13T12:25:32.179Z",
    "updatedAt": "2026-04-13T12:25:32.179Z",
    "__v": 0
}
```

http://localhost:3000/api/group/create

Save

POST http://localhost:3000/api/group/join Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

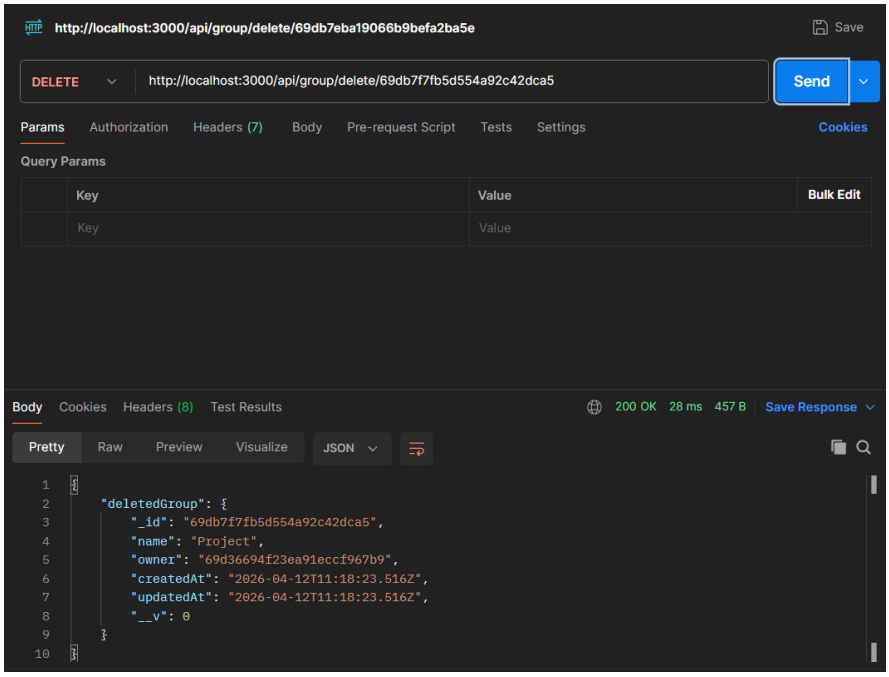
none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
    "inviteCode":"RTY32H23G"
}
```

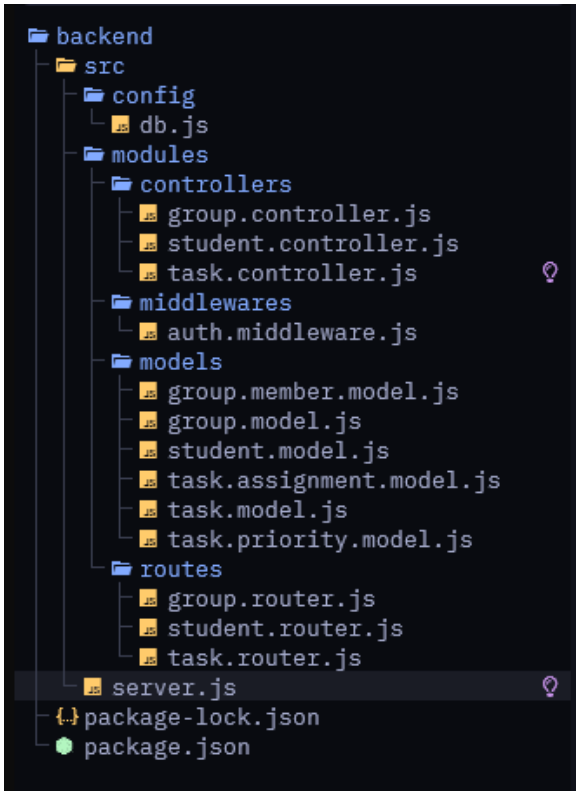
Body Cookies Headers (8) Test Results 200 OK 28 ms 491 B Save Response

Pretty Raw Preview Visualize JSON

```
{
    "member": {
        "groupId": "69db7eba19066b9befa2ba5e",
        "studentId": "69d36694f23ea91eccf967b9",
        "role": "member",
        "_id": "69dce0ec40accb718f04916d",
        "createdAt": "2026-04-13T12:26:20.320Z",
        "updatedAt": "2026-04-13T12:26:20.320Z",
        "__v": 0
    }
```



IV. Backend Folder Structure



V. Group Member Contributions

Member	Role	Contribution
Jay Lloyd Chicogo	Backend Developer	He built and maintains the server side of the system by setting up the Express server, creating routes, controllers, and models, and ensuring that all data processing and API responses work correctly and align with the needs of the frontend.
John Nelson Andes	Database Manager	He manages the database by creating a Mongoose models, establishing and maintaining the MongoDB connection, and ensuring that all data is properly structured, stored, and retrieved in a reliable and efficient way.
Jay Lloyd Chicogo	Github Manager	His job is to oversee the project repository by organizing the folder structure, commiting code consistently with clear and meaningful messages, and ensuring that all contributions are properly tracked and synchronized among team members.
Mico Malana	Documentation Officer	He handled the project documentation by capturing screenshots, explaining the functionality of each route, and preparing a clear and organized lab report that accurately reflects the system and its features.
Mico Malana	Project Manager	Monitors the overall progress of theproject by reviewing each assigned

		task, ensuring that all contributions are aligned with the project goals and requirements, and checking that the team stays on track with deadlines and expected outcomes.